

**EARLY DISEASE DETECTION USING
MORPHOLOGICAL ANALYSIS OF CELLS WITH DEEP
LEARNING**

**A INTERNSHIP PROJECT REPORT SUBMITTED IN
PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
THE AWARD OF THE DEGREE OF**

**BACHELOR OF TECHNOLOGY
IN**

COMPUTER SCIENCE AND ENGINEERING – DATA SCIENCE

BY

K. SHRUTHI	23071A6727
K. ANUROOP REDDY	23071A6728
M. AKSHITH REDDY	23071A6733
M. VAMSHI	23071A6734

UNDER THE GUIDANCE OF

Mrs. D. SREEVIDYA

Assistant Professor, Department of CSE-(CyS,DS) and AI&DS



**DEPARTMENT OF CSE-(CyS, DS) and AI&DS
ENGINEERING**

VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
Vignana Jyothi Nagar, Bachupally, Hyderabad 500118, Telangana India

May 2026



VNRVJIET

**VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF
ENGINEERING & TECHNOLOGY**

AUTONOMOUS INSTITUTION

DEPARTMENT OF CSE-(CyS, DS) and AI&DS ENGINEERING

CERTIFICATE

This is to certify that the Internship Project titled “**EARLY DISEASE DETECTION USING MORPHOLOGICAL ANALYSIS OF CELLS WITH DEEP LEARNING**” is a record of bona fide work submitted by:

K. SHRUTHI	23071A6727
K. ANUROOP REDDY	23071A6728
M. AKSHITH REDDY	23071A6733
M. VAMSHI	23071A6734

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **CSE-DS** of **Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R22 Academic Regulations**. The work has been carried out under our guidance and supervision at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**. The report is original, free from plagiarism, and complies with the Institute guidelines, including those related to academic ethics and AI-generated content.

Signature of Supervisor

Mrs. D. Sreevidya
Assistant Professor
Department of CSE- (CyS,
DS) and AI&DS
VNR VJIET, Hyderabad

Signature of HOD

Dr. G. Ramesh Chandra
Professor & I/C HOD
Department of CSE- (CyS,
DS) and AI&DS
VNR VJIET, Hyderabad

APPROVAL CERTIFICATE

Internship Project evaluation for the project titled “**EARLY DISEASE DETECTION USING MORPHOLOGICAL ANALYSIS OF CELLS WITH DEEP LEARNING**”, being submitted by **Ms. K. SHRUTHI**, Regd. No. **23071A6727**, **Mr. K. ANUROOP REDDY**, Regd. No. **23071A6728**, **Mr. M. AKSHITH REDDY**, Regd. No. **23071A6733**, **Mr. M. VAMSHI**, Regd. No. **23071A6734**, has been conducted on _____ and the work is approved for the award of the degree of **BACHELOR OF TECHNOLOGY (B.Tech.)** in **CSE-DS**, under VNRVJIET B.Tech. R22 Academic Regulations.

INTERNAL EXAMINER

EXTERNAL EXAMINER



VNRVJIET

**VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF
ENGINEERING & TECHNOLOGY**

AUTONOMOUS INSTITUTION

DEPARTMENT OF CSE-(CyS, DS) and AI&DS ENGINEERING

DECLARATION

We, the undersigned, declare that the Internship Project titled “**EARLY DISEASE DETECTION USING MORPHOLOGICAL ANALYSIS OF CELLS WITH DEEP LEARNING**” has been carried out by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.) in CSE-DS of Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R22 Academic Regulations**, at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

We further declare that this work is original, has not been submitted previously for any degree or similar title, and complies with the Institute’s plagiarism and academic ethics guidelines, including AI-generated content. The contributions of all team members are clearly indicated in the report.

PLACE: HYDERABAD

DATE: _____

Signature of the Student(s)

K. SHRUTHI

23071A6727

K. ANUROOP REDDY

23071A6728

M. AKSHITH REDDY

23071A6733

M. VAMSHI

23071A6734

ACKNOWLEDGEMENT

Firstly, we would like to express our immense gratitude towards our institution VNR Vignana Jyothi Institute of Engineering and Technology, which created a great platform to attain profound technical skills in the field of Computer Science, thereby fulfilling our most cherished goal.

We are very much thankful to our Principal, **Dr. Chenna Keshava Rao**, and our I/C Head , **Dr. G. Ramesh Chandra**, for extending their cooperation in doing this project within the stipulated time.

We extend our heartfelt thanks to our guide, **Mrs. D. Sreevidya** for her enthusiastic guidance throughout our project.

Last but not least, our appreciable obligation also goes to all the staff members of the Computer Science & Engineering department of CSE- (CyS, DS) and AI&DS and to our classmates who directly or indirectly helped us.

Ms. K. SHRUTHI	23071A6727
Mr. K. ANUROOP REDDY	23071A6728
Mr. M. AKSHITH REDDY	23071A6733
Mr. M. VAMSHI	23071A6734

ABSTRACT

Early detection of blood disease is becoming increasingly essential in the current healthcare context. There is an increase in the number of blood diseases hence the need for rapid diagnosis. Diseases such as Acute Lymphoblastic Leukemia (ALL), Malaria, and Sickle Cell Anemia can result in severe illnesses if not detected early. The conventional technique for the diagnosis involves examining pictures of blood smears through the microscope. This conventional technique depends on expertise in the field of medicine, which makes it inefficient and time-consuming.

The manual examination of blood smear images leads to inconsistencies in interpretation of results. The delay in detection may have negative effects on the treatment results and mortality rate. There is therefore the need to come up with a mechanism for automating the process of diagnosing diseases from images of blood smear. This would help improve the quality and efficiency of diagnosis.

This project seeks to design a deep learning algorithm that categorizes four classes of blood cell diseases such as Leukemia, Malaria, Sickle Cell Anemia, and normal blood cells. The deep learning algorithm will incorporate transfer learning and image-based morphology in order to detect abnormal cellular pattern automatically. Explainable AI will also be incorporated in the algorithm to improve predictions' interpretability.

Key words: Deep learning, hematological disorders, blood smear images, EfficientNetB0, transfer learning, Grad-CAM, multi-class classification.

LIST OF FIGURES

S.No.	Figure No.	Figure Description	Page No.
1	4.1.1	Class diagram	11
2	4.1.2.1	Use case diagram of the Model	12
3	4.1.3.1	Sequence diagram of the Model	13
4	4.1.3.2	Sequence diagram of the Proposed System	14
5	4.1.4	Activity diagram	15
6	4.2	Work flow diagram	17
7	5.1	Proposed Integrated Predictive modeling framework	18
8	6.1.1	Code Snippet of Dataset Structuring	20
9	6.2.1	Code Snippet of Data Preprocessing	21
10	6.3.1	Code Snippet of Model Building	21
11	6.4.1	Code Snippet of Model Training	22
12	6.5.1	Code Snippet of Model Loading	22
13	6.5.2	Code Snippet of Uploading Image	23
14	6.5.3	Code Snippet of Prediction	23
15	6.5.4	Code Snippet of Convolution Layer	24
16	6.5.5	Code Snippet of Gradcam	24
17	6.5.6	Code Snippet of Heatmap Overlay	25
18	6.5.7	Code Snippet of User Interface	25
19	7.2.1	Black Box Testing	28
20	7.2.2	Grey Box Testing	29
21	7.2.3	White Box Testing	30
22	7.3.1.1	Image Upload Interface	35
23	7.3.1.2	Prediction Result and Confidence Score	36
24	7.3.1.3	Class Probability Distribution	37
25	7.3.2.1	Input Blood smear image for sickle cell	38

26	7.3.2.2	Prediction Result and Confidence score	39
27	7.3.2.3	Bar plot results	40
28	8.1.1	Class wise performance Metrics	42
29	8.1.2	Confusion Matrix	43
30	8.1.3	ROC Curve for Multi Classification Model	44
31	8.1.4	Precision Recall Curve Analysis	45
32	8.1.5	Grad-CAM Based Model Interpretation	46
33	8.1.6	Training and validation accuracy and loss curves across epochs	47

TABLE OF CONTENTS

<i>Acknowledgements</i>	v
<i>Abstract</i>	vi
<i>List of Figures</i>	vii, viii
Chapter-1: Introduction	1-2
Chapter-2: Literature Survey / Existing Systems	3-6
2.1 Existing Systems	3-4
2.1.1 Some of the Existing Models	3
2.1.2 Drawbacks of the Existing System	4
2.2 Literature Survey	5-6
Chapter-3: Software Requirements	7-8
3.1 Functional Requirements	7
3.2 Non-functional Requirements	7-8
Chapter-4: Software Design	9-17
4.1 UML Diagram	9-15
4.1.1 Class Diagram	11
4.1.2 Use case Diagram	12
4.1.3 Sequence Diagram	13-14
4.1.4 Activity Diagram	15
4.2 Work Flow Diagram	16-17
Chapter-5: Proposed System	18-19
Chapter-6: Coding	20-25
6.1 Dataset Structuring and Class Organisation	20
6.2 Data Preprocessing	20-21
6.3 Model Building	21
6.4 Model Training	22
6.5 Model Loading	22-25
Chapter-7: Testing	26-40
7.1 Testing Types	26-29
7.1.1 Manual Testing	26
7.1.2 Automated Testing	27
Software Testing Methods	27-30
7.1.3 Black Box Testing	27-28

7.1.4 Gray Box Testing	28-29
7.1.5 White Box Testing	29-30
7.2 Testing Levels	30-35
7.2.1 Non-functional Testing	30-32
7.2.1.1 Performance Testing	30
7.2.1.2 Stress Testing	31
7.2.1.3 Security Testing	31
7.2.1.4 Portability Testing	31
7.2.1.5 Usability Testing	32
7.2.2 Functional Testing	32-35
7.2.2.1 Integration Testing	33
7.2.2.2 Regression Testing	33
7.2.2.3 Unit Testing	34
7.2.2.4 Alpha Testing	34
7.2.2.5 Beta Testing	34
7.3 Test Cases	35-40
7.3.1 Testcase-1	35-37
7.3.2 Testcase-2	37-40
Chapter-8: Output Screens/ Results	41-47
Chapter-9: Conclusions and Further Work	48-50
9.1 Conclusion	48
9.2 Summary	48-49
9.3 Further Work	49-50
References	51-52
Plagiarism Report	53
AI Detection Report	54
Show and Tell	55

CHAPTER 1

INTRODUCTION

Early diagnosis and treatment of blood disorders play a crucial role in the outcome for the patients and can dramatically influence recovery rate and overall well-being. Blood disorders like Acute Lymphoblastic Leukemia (ALL), Malaria and Sickle cell Anemia are prevalent causes of morbidity and mortality worldwide. Typically these disorders have distinct features, and are characterized by altered morphology of blood cells under microscope; Hence diagnosis is based on examining blood smears. Manual analysis under microscope is time-consuming and requires highly skilled personnel.

Currently, Hospitals and Clinics are facing a surge in number of blood disorder cases but there is scarcity of expert Hematologist and pathologist to make quick and efficient diagnosis. No machine has the stamina to work for hours and detect abnormalities from blood smears like the trained eyes, thus can cause human errors and inconsistency, especially when they have to go through multiple blood slides and also where a trained expert is hard to find. Hence, it has always been the goal to build automated systems.

Deep learning has evolved as one of the powerful tool to analyze medical images, they can effectively learn minor variations in raw images that human experts may neglect. For example, Convolutional Neural Network (CNN) could pick up variations in cell size, cell shape, cell texture, nucleus features, even detecting microscopic parasites which a less experienced eye could miss. Since they learn in layers, CNNs are proficient in detecting minor anomalies not normally taken care of in typical image analysis methods.

In Deep Learning, transfer learning method is particularly of value in medical context when labeled datasets are often small. Instead of training a model from scratch, a pre-trained model from a large dataset like EfficientNet, ResNet, or DenseNet has the advantage that it already knows what general feature to look for and can then adapt to medical images more rapidly and efficiently.

The project that we aimed is to identify blood disorders at an early stage by developing a deep learning model to analyze blood cell images. It's designed as a multi class classifier that can detect Leukemia, Malaria, Sickle Cell Anemia and Normal Blood Cells. The deep learning model used here is an EfficientNetB0 as the classifier and feature extractor. The pipeline of the project would involve the steps that is Collecting data, processing it, Merging datasets, Extracting Features, Classifying cells, Testing the model and analyzing what the model has

predicted.

Common Detection Perspectives

1. Morphology-centric Perspective

- Focuses on identifying abnormalities in blood cell shape, size, and structure
- Useful for detecting sickle-shaped RBCs, enlarged nuclei, and parasite presence

2. Disease-centric Perspective

- Emphasizes disease-specific visual characteristics
- Helps distinguish between leukemia, malaria, and normal cells

3. Explainability-centric Perspective

- Uses Grad-CAM to visualize regions influencing model predictions
- Improves trust and interpretability in medical diagnosis

An iterative and scalable approach will be employed within the framework in order to guarantee that the classifier is highly accurate and reliable. The model will incorporate Grad-CAM explainability in order to make its decision-making process transparent. In turn, it increases the usefulness of the developed system since doctors can verify the results generated by the model.

Thus, the primary objective of this project is to provide clinicians with an intelligent, automated, and explainable solution to their problems. It is intended to minimize their efforts and decrease the number of errors during diagnosis while helping to detect blood diseases efficiently.

CHAPTER 2

LITERATURE SURVEY / EXISTING WORK

2.1 EXISTING WORK

2.1.1 Some of the Existing Models

1. Malaria Detection using CNN:

Some scientists have used CNNs to develop models that detect malaria using blood smear pictures. These systems aim to find parasite-infected red blood cells by learning about variations in texture and color. Although these models reach high accuracy, they only classify malaria.

2. Leukemia Detection Systems:

Leukemia detection is something that deep learning architectures like ResNet, DenseNet and VGGNet are really good at. These leukemia detection systems look at things like the size of the nucleus and the texture of the chromatin in blood cells. They also check for irregularities, in the cell boundary. This helps them figure out if a white blood cell is abnormal or not. Leukemia detection systems that use ResNet, DenseNet and VGGNet are able to do this by analyzing these things.

3. Sickle Cell Detection Models:

Traditional methods and machine learning have been used to detect sickle cell anemia. These models mostly look at the shape of blood cells to identify sickle-shaped ones using simple features, like the shape and outline of the sickle-shaped red blood cells and focus on finding crescent-shaped red blood cells.

4. Transfer Learning-based Medical Imaging Systems:

Recent systems use things like EfficientNetB0 and ResNet50 and DenseNet121 to classify diseases. These models are more efficient. They require less training time. Moreover, they work only on certain kinds of diseases such as cancers and diabetes. It means that these models, EfficientNetB0, ResNet50, and DenseNet121, are useful for disease classification.

2.1.2 DRAWBACKS OF THE EXISTING SYSTEM

1. Single-Disease Classification

- Most existing models are designed to detect only one specific disease such as malaria or leukemia.
- These systems cannot classify multiple hematological disorders simultaneously.
- This limits their practical use in real-world diagnostic environments.

2. Lack of Integrated Multi-Class Framework

- Existing systems lack a unified framework for detecting multiple blood-related diseases.
- Separate models are often required for different diseases, increasing complexity.
- This reduces scalability and efficiency in clinical applications.

3. Poor Performance on Imbalanced Datasets

- Medical datasets often have unequal distribution among disease classes.
- Rare diseases such as sickle cell anemia usually have fewer samples.
- Many existing models fail to perform well on minority classes.

4. Limited Explainability and Visualization

- Several systems provide predictions without explaining the decision process.
- Lack of visualization makes it difficult for clinicians to trust the model.
- Absence of techniques such as Grad-CAM reduces interpretability.

5. Dependency on Large Labeled Datasets

- Deep learning models require a large number of labeled images for training.
- Collecting and annotating medical images is expensive and time-consuming.
- This makes training difficult in limited-resource scenarios.

6. Challenges in Real-World Deployment

- Existing systems are often tested only in controlled environments.
- They may not generalize well to clinical images from different labs or hospitals.
- Deployment challenges include variability in image quality and staining methods.

2.2 LITERATURE SURVEY

Much research has been conducted concerning automated medical image classification, especially for the diagnosis of hematological diseases. Since artificial intelligence has advanced greatly with the help of computing power, the majority of these studies have considered applying computer vision technologies on images of blood smears.

Early research considered identifying the presence of malaria parasites in blood cells via the usage of Convolutional Neural Networks (CNN). The CNN system was capable of identifying anomalies in both the color and morphology of the cell and even spotting the parasite. This research proves that deep learning-based solutions are significantly more effective in diagnosing morphological and textural abnormalities of cells than conventional approaches.

Other scholars used the method of convoluted networks for the diagnosis of leukemia by examining images of white blood cells. The CNN system proved to be more efficient in identifying nuclei, abnormally shaped cells and atypical chromatin structures compared to such approaches as SVM and RF classifier. Some models, like VGGNet, ResNet and DenseNet are used a lot because they are really good at finding features in pictures.

At first researchers trying to detect sickle cell anemia used image processing techniques like edge detection and thresholding. They looked for crescent-shaped blood cells by measuring how eccentric they were and how irregular their borders were.. These methods were not very good and were easily confused by noise in the pictures or by differences in how the pictures were taken.

One of the recent research trends has involved the development of transfer learning techniques, whereby models pre-trained on vast amounts of images such as EfficientNet, ResNet50, DenseNet121, and MobileNet are further trained to classify images in medical scenarios. Such models demonstrate better precision, shorter training periods, and are even efficient when employed with limited data due to their ability to leverage information extracted during training from large datasets such as ImageNet.

In addition, there is an increasing trend of ensuring that artificial intelligence models are made more explainable, thus allowing doctors to understand the reasoning behind the results. For instance, researchers employ Grad-CAM and heatmap-based techniques for visualizing model

outputs and improving their transparency and reliability. This allows doctors to understand what parts of the image were considered by the AI model while making decisions, which leads to improved reliability.

While all these developments, the majority of the works done in this field only focus on detecting one disease at a time without having a systematic approach that is able to detect different hematological diseases altogether. This study draws from the existing literature on this topic and proposes a single classifier system that is capable of diagnosing various types of leukemia, malaria, sickle cell anemia, and normal blood cells through Grad-CAM visualization. The classifier system will be multi-class deep learning based which will provide a platform that can diagnose multiple hematological diseases at the same time. Medical image classification systems use learning models to interpret medical images of blood smears such as leukemia, malaria, and sickle cell anemia.

CHAPTER 3

SOFTWARE REQUIREMENTS

3.1 Functional Requirements

- **Image Upload:**
The system must allow users to upload microscopic blood smear images for analysis and disease prediction.
- **Image Preprocessing:**
The system must preprocess uploaded images by resizing, normalization, and formatting them according to the model requirements.
- **Disease Classification:**
This entails that the system classifies the uploaded image of the blood smear into one of the categories of diseases; that is, Leukemia, Malaria, Sickle Cell Anemia, or Normal Blood Cells.
- **Prediction Confidence:**
The system must display the prediction probability or confidence score for the detected disease.
- **Grad-CAM Visualization:**
The system must generate Grad-CAM heatmaps to highlight the regions of the image influencing the prediction.
- **Result Display:**
The system must display the predicted disease label along with the visualization output in a user-friendly format.
- **Model Management:**
The system must load and use the trained deep learning model for prediction.
- **Dataset Handling:**
The system must support training and validation on multi-class blood smear image datasets.

3.2 Non-functional Requirements

- **Accuracy:**
High accuracy and consistency in predictions and classifications should be offered by the system.
- **Performance:**
The system should be able to produce predictions within a few seconds of uploading images.

- **Reliability:**
The output should be reliable with as few mistakes as possible.
- **Usability:**
The user interface is supposed to be easy to use and simple for the doctors and researchers.
- **Scalability:**
This should be designed such that it is able to accommodate future growth to include other types of blood-related disorders.
- **Maintainability:**
The system must be upgradable and maintainable in order to enhance it further in the future.
- **Interpretability:**
Explanation services for AI should be provided through the Grad-CAM visualization technique.
- **Portability:**
The solution must be scalable on various platforms, such as Google Colab, local machines, and Web Apps using Streamlit.

CHAPTER 4

SOFTWARE DESIGN

4.1 UML Diagrams

The following UML diagrams are effective tools that enable software designers, developers, and stakeholders to graphically depict various elements of the system or process involved in the software development process.

4.1.1 Class Diagram

The importance of the class diagram lies in its role in illustrating and structuring the design of the disease detection system. This diagram depicts how the entities (classes) within the system are interconnected with each other. The various classes within our model include:

Classes:

Blood Image Class

Represents the uploaded blood smear image.

Attributes:

- image_id : int
- image_path : string
- image_size : tuple

Methods:

- uploadImage()
- preprocessImage()
- resizeImage()

Prediction Model Class

Represents the deep learning model.

Attributes:

- model_name : string
- model_type : string
- accuracy: float

Methods:

- loadModel()
- predictDisease()
- generateConfidence()

GradCAM Class

Represents explainability module.

Attributes:

- heatmap: image
- overlay: image

Methods:

- generateHeatmap()
- overlayOnImage()

UserInterface Class:

Represents Streamlit UI.

Attributes:

- uploaded_file
- prediction_result
- confidence_score

Methods:

- displayUI()
- showPrediction()
- showGradCAM()

Relationships

- One Blood Image → One Prediction Model
- One Prediction Model → One GradCAM
- One User Interface → Multiple predictions

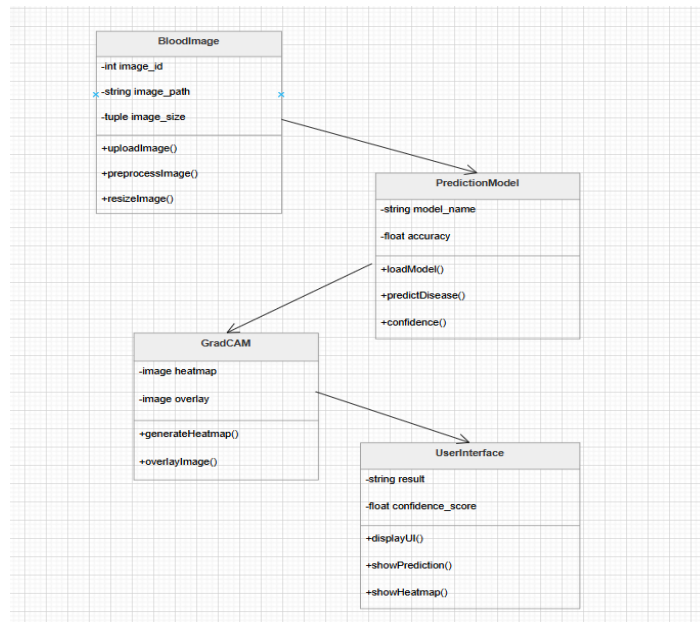


Fig 4.1.1. Class Diagram

4.1.2 Use case Diagram

This is a use case diagram that demonstrates the interaction between the User and the system

Actor

- User

Use Cases

- Upload Blood Smear Image
- Preprocess Image
- Predict Disease
- Generate Grad-CAM
- Display Result
- Show Confidence Score

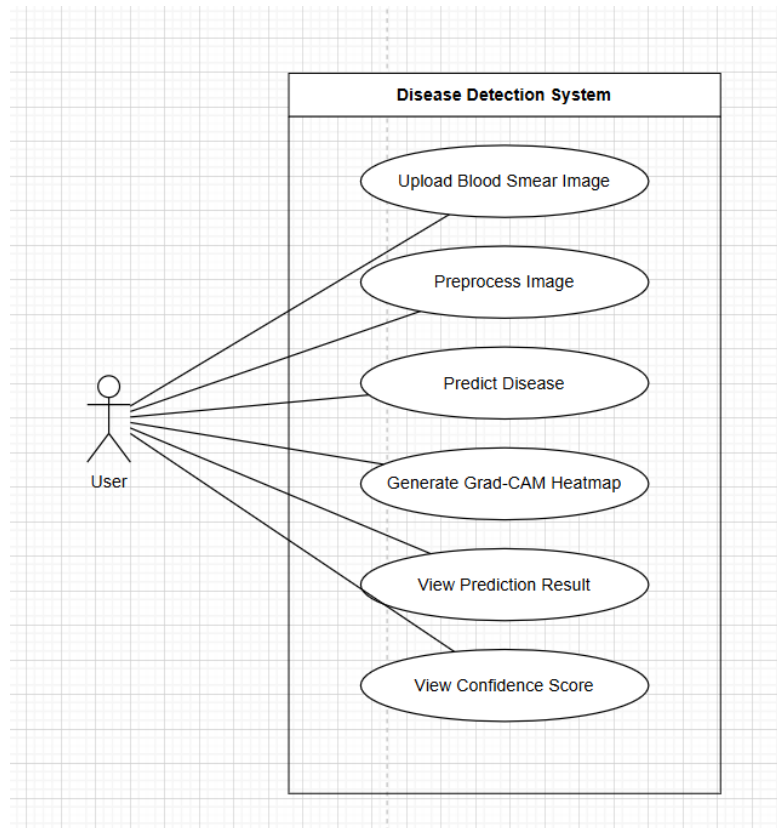


Fig 4.1.2.1. use case diagram of the Model

4.1.3 Sequence Diagram

This sequence diagram demonstrates the communication between the User and a system component, Disease Detection System, that helps in the process of uploading an image of a blood smear, prediction of the disease, and creation of Grad-CAM, where Prediction Model serves as a processing component.

Uploading and Prediction Process:

- The User starts the procedure by submitting an image of the blood smear via the user interface using the `uploadImage()` function.
- This `DiseaseDetectionSystem` module now connects with the Prediction model to prepare the image for processing through resizing and normalizing. This process is depicted by the `preprocessImage()` operation.
- Once preprocessed, the Prediction Model classifies the disease based on the execution of the `predictDisease()` function, which determines if the input is either Leukemia, Malaria, Sickle Cell Anemia, or normal blood cells
- Once the prediction is completed, the **Prediction Model** sends the prediction result and confidence

score back to the **DiseaseDetectionSystem**.

- The system then communicates with the **GradCAM Module** to generate a heatmap visualization, which is represented by the generateHeatmap() action.
- After generating the heatmap, the **GradCAM Module** overlays the heatmap on the original blood smear image and sends the visualization back to the **DiseaseDetectionSystem**.
- Finally, the **DiseaseDetectionSystem** updates the local interface state and displays the prediction result along with the Grad-CAM visualization to the **User**, which is denoted by the displayResult() action.

Validation and Output Process:

- The system performs validation by checking the prediction confidence score and ensuring that the output meets the required threshold.
- Upon successful validation, the final result is displayed to the user.
- If the confidence score is low, the system may indicate uncertain prediction or request further analysis

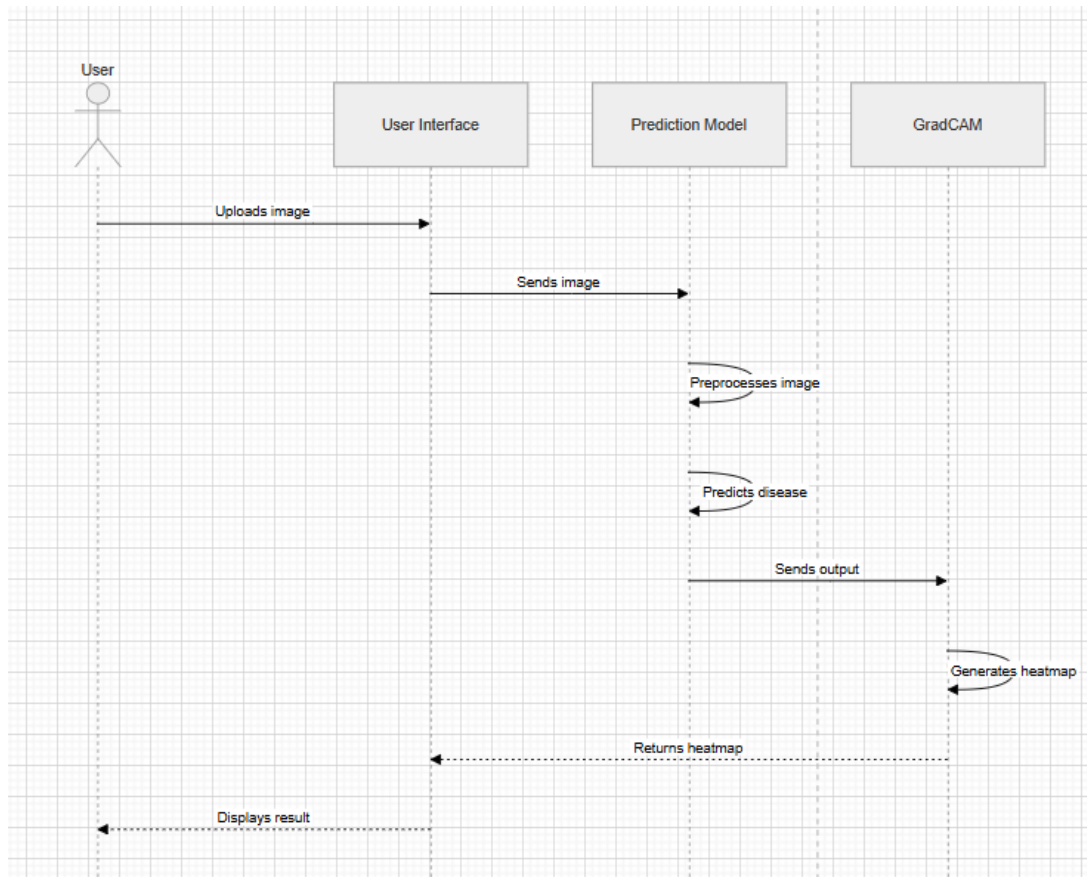


Fig 4.1.3.1. Sequence diagram of the Model

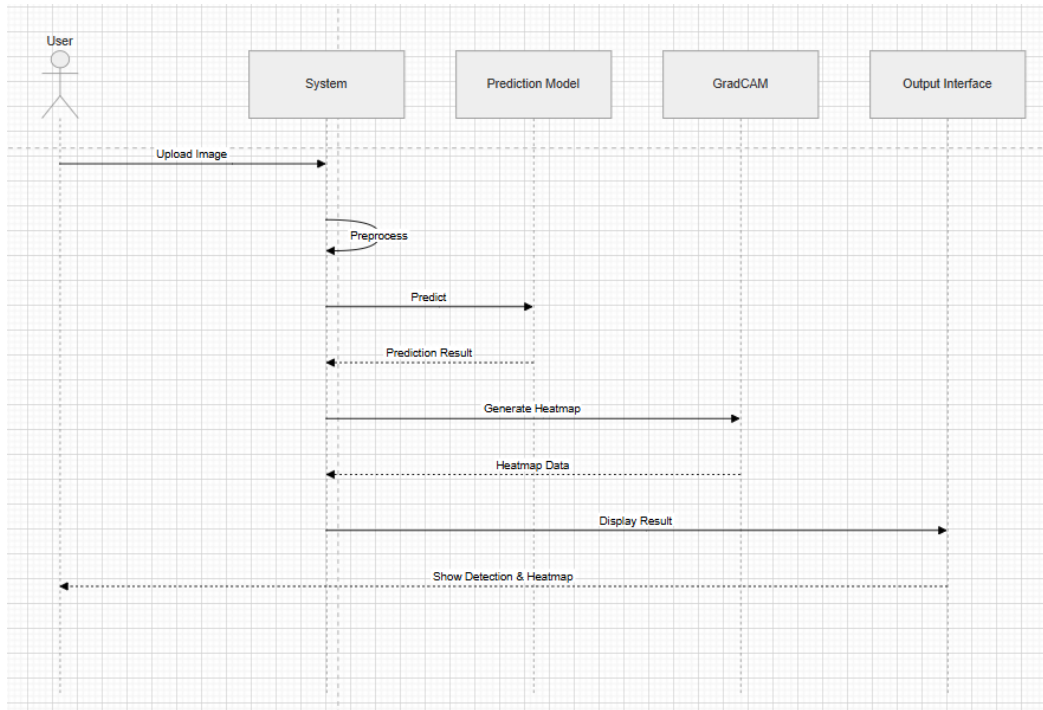


Fig 4.1.3.2. Sequence diagram of the Proposed system

4.1.4 Activity Diagram

Activity diagram is a tool for visualizing the workflow of the **User and the Disease Detection System** during the disease prediction phase. It helps in understanding the steps involved in uploading the blood smear image, processing it through the deep learning model, and displaying the final disease prediction along with the Grad-CAM visualization. It also shows the flow from one activity to the next, including the decision-making process.

Start of the Process:

The activity begins with the **User accessing the disease detection system** and uploading a microscopic blood smear image.

Prediction Process:

- The **User starts by uploading a blood smear image** to the system. This involves selecting the image through the user interface from the device.
- Then comes the pre-processing phase, where the image is resized, normalized, and prepared for the deep learning algorithm.
- First, the Prediction Model is invoked, and then the feature extraction process is carried out with the help of the pre-trained EfficientNet model.
- After feature extraction, the **system predicts the disease class**. This includes classifying the image into

one of the predefined categories: **Leukemia, Malaria, Sickle Cell Anemia, or Normal blood cells**.

- Following the prediction, the **Grad-CAM module generates a heatmap** to highlight the important regions of the image that influenced the prediction.
- Finally, the **system displays the prediction result, confidence score, and Grad-CAM visualization** to the user.

Validation / Decision Step:

- The system checks whether the **prediction confidence score meets the required threshold**.
- If the confidence is low, the system may indicate uncertain prediction or request re-evaluation.
- If the confidence is sufficient, the final result is displayed successfully.

End of the Process:

After the prediction result and visualization are displayed, the activity concludes.

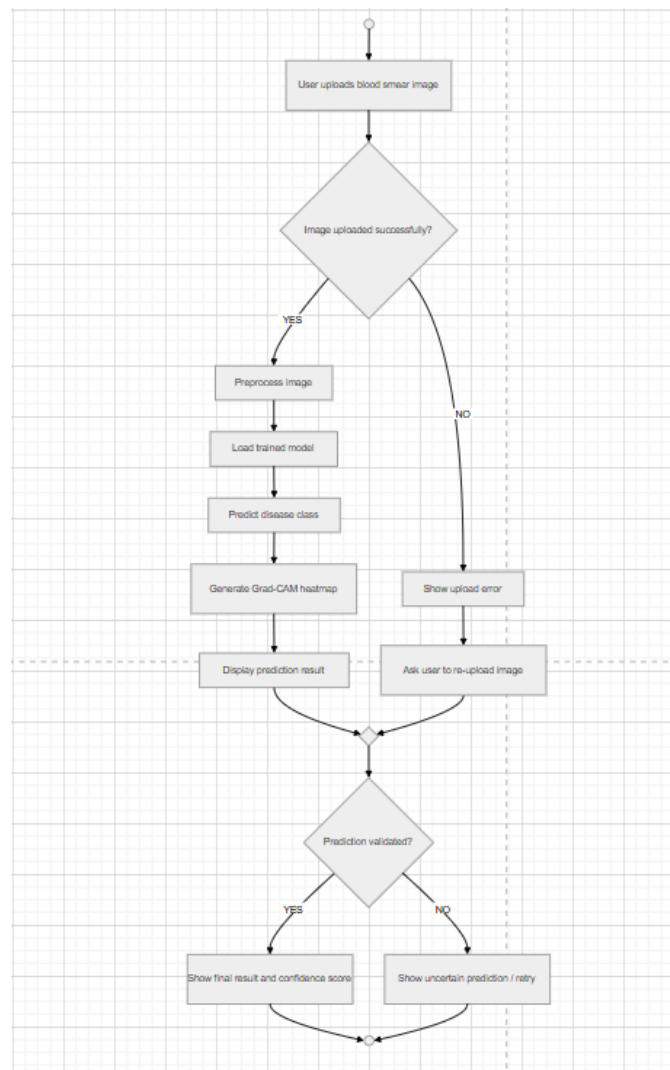


Fig 4.1.4. Activity diagram

4.2 Work Flow Diagram

So the workflow diagram depicts the entire detection of blood diseases using deep learning in a looped manner, from the very first point of user logging into the application and uploading a blood smear image to receiving the result. First, the blood smear image is acquired after the user successfully logs in and uploads the blood smear image. Then, data acquisition and preprocessing phase begins; where, the image is collected and normalized after resizing it and it is ready for prediction.

Prediction phase; it first loads the trained EfficientNetB0 deep learning model. The system analyzes and divides the photograph into cell shape, size of the nucleus and also presence of any parasites, abnormal structure and other characteristic from it. Based on these features it predicts what the given blood smear images depicts. Hence it predicts if it is Leukaemia or Malaria or Sickle cell anemia or healthy.

After, a prediction is done it reaches the visualization and validation phase. The Grad-CAM module takes the picture, and draws a heat map highlighting the regions of the picture which resulted in the prediction made. It also confirms that the prediction made has some confidence behind it and it's not merely a chance.

The system then displays the results to the user itself, through the interface; where the diagnosis of the blood diseases, along with its confidence level and also the Grad-CAM map.

The last part of this workflow diagram is the feedback and review. Here the human/analyst can confirm or refute the detection; check if more data is needed or better training has to be done. And then if there are some new images/diseases they can be input back into the loop.

CHAPTER 5

PROPOSED SYSTEM

This system uses deep learning to identify and categorize diseases associated with blood using images of blood smear under a microscope. It is performed in different steps while passing from stage to stage in order to perform the image processing, disease identification, display visualizations, and achieve improvements in predictions. The system undergoes 3 stages: preprocessing stage, classification stage, and verification and visualization stage. Each stage has its share in improving the accuracy and reliability of disease detection.

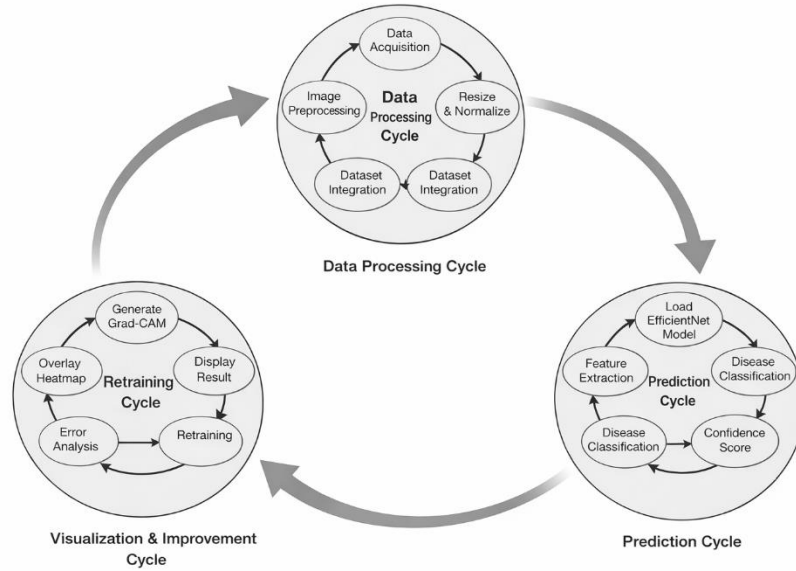


Fig 5.1. Proposed Integrated prediction modeling framework

All operations begin with data processing. During this process, microscopic blood smear images are collected from various data sets, where some of them contain Leukemia, Malaria, Sickle Cell Anemia images, while the rest have normal blood cells images. After the images have been collected, they are preprocessed by means of resizing, normalization, and conversion to a format acceptable for EfficientNet. Having done this step, the images are categorized to pave the way for further supervised learning and multi-classification.

Afterwards, let us move on to the prediction stage. The preprocessed images will be fed into a

model called EfficientNetB0 that will analyze the images and detect cell boundaries, nuclei structures, parasite infections, and any abnormalities in the images. The detected features then go to fully connected layers to determine which of the four classes is related to an image. As a result of predictions, confidence scores for all classifications are produced. The purpose of the prediction is to make the disease detection fast and accurate due to transfer learning and feature extraction methods.

The last part is visualization and validation. With this step, the system becomes more reliable and easy-to-understand. By applying the Grad-CAM method, the system develops heatmaps to highlight the areas of images on which its predictions are based. By overlaying heatmaps onto the original images, users can get insights into what the model sees. The system evaluates the level of confidence and ensures accuracy for each disease type.

The final step in our framework is a continuous improvement and retraining process. Over time, the system receives newly available data about blood cell samples, incorrect classifications of images, as well as any new disease categories identified by medical researchers. Therefore, our system undergoes retraining from time to time, which makes it capable of performing more effectively in terms of classification.

In conclusion, our framework provides an efficient way to detect blood diseases using artificial intelligence. It is fully automated, scalable, and most importantly, explainable to doctors because of heatmaps. In healthcare, it helps reduce the workload associated with the manual analysis of microscopic images.

CHAPTER 6

CODING

The system to find hematology disease works in steps. These steps are loading the model getting the images ready making a prediction and explaining it with something called Grad-CAM. Each step is like a part and has its own code in the system. The system has parts, like loading the model and each part is separate. The hematology disease detection system has steps.

6.1 Dataset structuring and class organization

In this part we consolidate several public available datasets and store them under a common directory which is usable for multi-class classification. In this case the classes are evident and can be thought of as malaria, leukemia, sickle cell anemia, and normal cell. For the malaria dataset, any image labeled as infected should be stored in the 'malaria' folder and an uninfected image in the 'normal' class folder. Similarly, for the leukemia images we both use cancerous images and healthy images, and assign the cancerous image to the 'leukemia' folder and the healthy image to the 'normal' class folder. The images belonging to the sickle cell disease folder all labeled as sickle cell should be saved under 'sickle'. All images from these categories would reside in the master directory 'multi_dataset', and each folder inside of it would be a unique class as shown in the image. This data structure can easily be passed into deep learning models available in software such as TensorFlow.

```
# Create unified dataset folders
os.makedirs("multi_dataset/malaria", exist_ok=True)
os.makedirs("multi_dataset/leukemia", exist_ok=True)
os.makedirs("multi_dataset/normal", exist_ok=True)
os.makedirs("multi_dataset/sickle", exist_ok=True)

# -----
# MALARIA DATASET
# -----
parasitized_path = "cell_images/cell_images/Parasitized"
uninfected_path = "cell_images/cell_images/Uninfected"

for file in os.listdir(parasitized_path):
    shutil.copy(os.path.join(parasitized_path, file), "multi_dataset/malaria")

for file in os.listdir(uninfected_path):
    shutil.copy(os.path.join(uninfected_path, file), "multi_dataset/normal")

# -----
# LEUKEMIA DATASET
# -----
leukemia_path = "C-NMC_Leukemia/training_data/fold_0/all"

for file in os.listdir(leukemia_path):
    shutil.copy(os.path.join(leukemia_path, file), "multi_dataset/leukemia")
```

Fig 6.1.1. Code Snippet of Dataset Structuring

6.2 Data Preprocessing

Blood smear images are preprocessed to resize a standard input size and to normalize pixels for consistency within the dataset. Rotation, flipping, and zooming are some of the data augmentation methods that are used to improve image variety. This makes the neural network more robust because overfitting is minimized by using these preprocessing methods.

```
train_generator = datagen.flow_from_directory(  
    "multi_dataset",  
    target_size=(224, 224),  
    batch_size=16,  
    class_mode='categorical',  
    subset='training'  
)  
  
val_generator = datagen.flow_from_directory(  
    "multi_dataset",  
    target_size=(224, 224),  
    batch_size=32,  
    class_mode='categorical',  
    subset='validation',  
    shuffle=False  
)  
  
print("Class indices:", train_generator.class_indices)
```

Fig 6.2.1. Code Snippet of Data Preprocessing

6.3 Model Building

Efficientnet-b0 model serves as the base model for extracting features since it performs better than others when it comes to classifying images. The pre-trained model utilizes transfer learning to identify relevant morphological characteristics including the structure, shape, and appearance of the cells in the blood smear images. Additional classification layers have been included to ensure that the multi-class classification is done accurately.

```

from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, models

base_model = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)  # ✅ FIXED
)

# Freeze base layers
for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.5)(x)
output = layers.Dense(4, activation='softmax')(x)

model = models.Model(inputs=base_model.input, outputs=output)

```

Fig 6.3.1. Code Snippet of Model Building

6.4 Model Training

Category Cross Entropy loss function has been used for the training of the model. This loss function is suitable for multi-class classification problem. Adam optimizer is used to update weights effectively in the training by minimizing the loss function. This will lead to quicker convergence and smoother learning as well as the improved classification accuracy.

```

from tensorflow.keras.optimizers import Adam
import tensorflow as tf

loss_fn = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)

model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss=loss_fn,
    metrics=['accuracy']
)

model.summary()

```

Fig 6.4.1. Code Snippet of Model Training

6.5 Model Loading

In the first step, the model is loaded into the system through TensorFlow. This model contains the learned weights of the neural network and its architecture. These models are saved in the .keras file format. This process initializes the model so that it is ready to predict disease from blood smear images.

```
# =====  
# ♦ LOAD MODEL  
# =====  
uploaded = files.upload()  
  
model = tf.keras.models.load_model(  
    "hematology_4class_model_cleaned.keras",  
    compile=False  
)  
  
print("✅ Model loaded successfully")
```

Fig 6.5.1. Code Snippet of Loading Pre-trained model

In this step, the input blood smear image is uploaded and preprocessed before it is fed into the model. Preprocessing is done to ensure that the input is compatible with the architecture of the EfficientNet.

```
# =====  
# ♦ UPLOAD IMAGE  
# =====  
uploaded = files.upload()  
  
img_path = list(uploaded.keys())[0]  
  
img = cv2.imread(img_path)  
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
  
plt.imshow(img)  
plt.title("Input Image")  
plt.axis("off")  
plt.show()
```

Fig 6.5.2. Code Snippet of Uploading Image

This stage carries out the classification process by utilizing the trained model. This model assigns probabilities to each disease category, and the category with the maximum probability is chosen as the final output.

```
# =====  
# ♦ PREDICTION  
# =====  
preds = model.predict(img_array)  
  
class_names = ['leukemia', 'malaria', 'normal', 'sickle']  
  
pred_class = class_names[np.argmax(preds)]  
confidence = np.max(preds)  
  
print(f"🧠 Prediction: {pred_class}")  
print(f"📊 Confidence: {confidence:.4f}")
```

Fig 6.5.3. Code Snippet of Prediction

For the creation of the Grad-CAM output, the network determines the last convolutional layer of the model. This will make sure that the method works with all sorts of models.

```
# =====  
# ♦ FIND LAST CONV LAYER  
# =====  
last_conv_layer_name = None  
  
for layer in model.layers[0].layers[::-1]:  
    if isinstance(layer, tf.keras.layers.Conv2D):  
        last_conv_layer_name = layer.name  
        break  
  
print("Last Conv Layer:", last_conv_layer_name)
```

Fig 6.5.4. Code Snippet of Convolutional Layer

Grad-CAM helps us understand which parts of the image contribute to the model's decision-making process. This increases our understanding and faith in the system.

```
def make_gradcam_heatmap(img_array, model, last_conv_layer_name):
    base_model = model.layers[0]

    grad_model = tf.keras.models.Model(
        inputs=model.input,
        outputs=[base_model.get_layer(last_conv_layer_name).output, model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        class_idx = tf.argmax(predictions[0])
        loss = predictions[:, class_idx]

    grads = tape.gradient(loss, conv_outputs)

    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]

    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)

    heatmap = tf.maximum(heatmap, 0)
    heatmap /= tf.reduce_max(heatmap) + 1e-8

    return heatmap.numpy()
```

Fig 6.5.5. Code Snippet of Gradcam

Heatmap is applied on the original image for emphasis of important areas which contribute towards the decision-making process of the model.

```
# =====
# ◆ OVERLAY
# =====
heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
heatmap = np.uint8(255 * heatmap)

heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)

superimposed_img = cv2.addWeighted(img, 0.6, heatmap, 0.4, 0)

plt.imshow(superimposed_img)
plt.title("Grad-CAM Overlay")
plt.axis("off")
plt.show()
```

Fig 6.5.6. Code Snippet of Heatmap Overlay

This is the last step where the model is incorporated into a user interface through Streamlit. This will enable users to input images and get predictions, as well as Grad-CAM explanations

```

st.title("🩸 Hematology Disease Detection with Grad-CAM")

uploaded_file = st.file_uploader("Upload Blood Cell Image", type=["jpg", "png", "jpeg"])

if uploaded_file is not None:
    image = Image.open(uploaded_file)
    img = np.array(image)

    st.image(img, caption="Uploaded Image", use_column_width=True)

    # Preprocess
    img_resized = cv2.resize(img, (224, 224))
    img_array = np.expand_dims(img_resized, axis=0)
    img_array = preprocess_input(img_array)

    # Prediction
    preds = model.predict(img_array)
    pred_class = class_names[np.argmax(preds)]
    confidence = np.max(preds)

    st.subheader(f"🩺 Prediction: {pred_class}")
    st.write(f"📊 Confidence: {confidence:.4f}")

    # Grad-CAM
    last_conv_layer_name = get_last_conv_layer(model)
    heatmap = make_gradcam_heatmap(img_array, model, last_conv_layer_name)

    heatmap = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)

    superimposed_img = cv2.addWeighted(img, 0.6, heatmap, 0.4, 0)

    st.image(superimposed_img, caption="🔥 Grad-CAM Visualization", use_column_width=True)

```

Fig 6.5.7. Code Snippet of Implementing user interface using Streamlit

CHAPTER 7

TESTING

Testing is an extremely important aspect to build a deep learning system in order to diagnose hematological diseases. This step helps us know the actual performance of our model, i.e., to know how good it is to produce stable and accurate outputs when it is fed with different image datasets. This becomes a very essential part when dealing with medical problems because every diagnosis must be accurate and clinically useful to the clinician. In this context, it validates the capability of the model to differentiate between the images for Leukemia, Malaria, Sickle Cell Anemia and normal cells and also to analyze over-fitting or error in classification, imbalanced data problems etc.

For the project that we are building, testing is the process to match our prediction with the true outcome for new image inputs and also find the performance of the model by calculating accuracy, precision, recall and f1-score.

7.1 TESTING TYPES

7.1.1 MANUAL TESTING

We ran manual tests to see how the system performs from both the user's and an expert's point of view. In this phase, a large set of sample images of blood smear, which have been categorized into various disease groups, were uploaded into the software system to verify whether the model correctly classified the images.

However, beyond determining accuracy in prediction, the confidence level of each prediction was carefully observed. Additionally, the heatmaps generated by the model were evaluated. As an example, when a leukemia image is inputted, the model must be able to identify the abnormal nuclei of the white blood cells. When a malaria image is inputted, the software must be able to point out the presence of parasitic infections in the red blood cells. In cases wherein the attention was misdirected, there might be issues that require improvement. Therefore, such a process can help determine whether the model's rationale is consistent with medical practitioners' expectations.

Testing the software through manual manipulation allowed for an accurate understanding of its usability, particularly with the incorporation of Streamlit interface into the software.

7.1.2 AUTOMATED TESTING

An analysis was performed on the validation data set to find out how effectively the model was able to perform. In other words, an unknown number of images of blood smears were shown to the model and it classified them.

Testing is done in order to be able to evaluate the model's performance with specific metrics and re-test it at any time in the future. For this study, accuracy, precision, recall, F1-Score, confusion matrix and loss were selected for analyzing the results for each category of the diseases.

The test images that were used were not included in the training set in any way. They were all completely new and unknown to the model, just as the real-world situation would be.

SOFTWARE TESTING METHODS

7.1.3 BLACK BOX TESTING

Black box testing is based on observing the behaviour of the system under various inputs and inspecting the results while not looking inside the workings of the system.

In our case, we provide a sample to the model and see which class is assigned to this image according to our data. However, we don't consider the process of determining the result because our concern is that black box testing imitates the work of people using the software – they provide images and receive results that should be correct.

That makes black box testing extremely helpful for testing purposes.

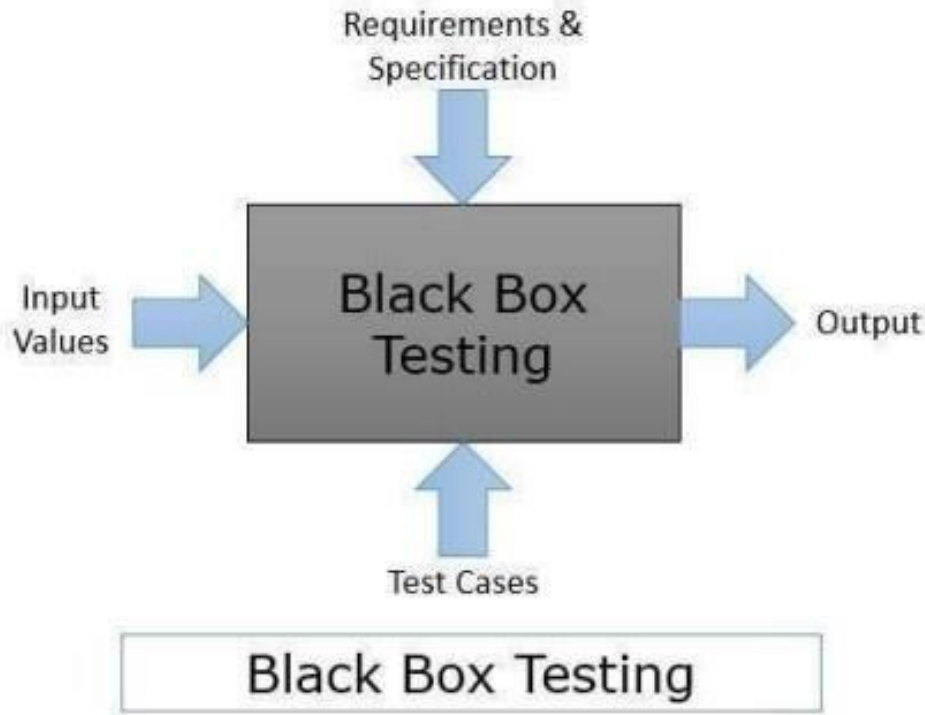


Fig 7.2.1 Black Box Testing

7.1.4 GRAY BOX TESTING

Gray box testing methodology combines aspects of black and white box testing. The tester has some knowledge of the inner workings of the software, but the emphasis of the test is placed on the validation of the system's outputs. It is highly useful when applied to deep learning models, as not only the output matters but also the rationale behind the model's choice of prediction.

In this project, gray box testing implies the analysis of the predicted class labels and certain inner states of the model. Among these inner states are the confidence scores (the probability of a particular disease assigned to the respective class), along with the visualization of the model's decisions, such as Grad-CAM or other attribution methods. The tester could further explore the convolutional feature maps generated at various stages of the processing.

This is where you start to see exactly how the system is recognizing different patterns—like cell shape, edges, the structure of the nucleus, or changes in texture. A visual summary of this approach appears in Figure 7.2.2.



Fig 7.2.2 Grey Box Testing

7.1.5 WHITE BOX TESTING

White box testing is essentially you getting "inside" the deep learning model and checking all the plumbing works correctly. So rather than just see whether it outputs the correct class, you want to be checking how it gets there; that is how it processes an input image, and the type of features it has learned to focus on.

For this project it involves you inspecting the internal working of the EfficientNetB0 architecture, examining its layers of convolution and features extraction, and inspecting the training and validation loss curves. The rise of the curves over batches and epochs indicate whether the model has learned correctly and is reaching a suitable minimum value for its loss function, or has got stuck.

This is also useful for detecting any larger problems in the model, such as overfitting and underfitting, and any learning issues where the learning is shaky or very slow or refuses to be settled down. It allows you to investigate the layers themselves and check that the correct morphology has been learned from the blood smear images.

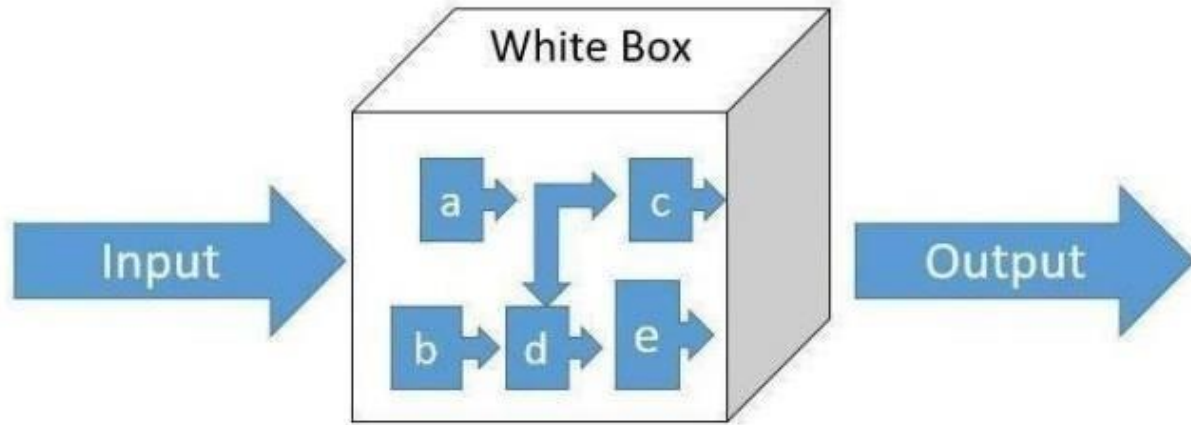


Fig 7.2.3. White Box Testing

7.2 TESTING LEVELS

7.2.1 NON-FUNCTIONAL TESTING

Non-functional testing involves the assessment of the dependability, performance, and accountability of the software from the point of view of user needs. The main purpose of the assessment is measuring performance as opposed to functionality based on specific KPIs. Functional testing is different from non-functional because the latter looks at what resides beyond functionality and is vital since customer satisfaction is difficult to attain.

7.2.1.1 PERFORMANCE TESTING

The purpose of Performance testing in this case is the determination of how quickly and efficiently the model performs during predictions. The tests will measure the amount of time it takes for the model to predict the disease from the input image.

Performance testing guarantees that the model is highly efficient without much delay. In addition, performance testing ensures that EfficientNetB0 achieves fast inference while being very accurate.

7.2.1.2 STRESS TESTING

Stress testing is performed to evaluate how the system behaves under high workloads and extreme input conditions.

In this project, this includes:

- Testing multiple image predictions continuously
- Running large batches of validation images
- Checking GPU memory usage
- Monitoring system response time

This helps ensure that the system remains stable even when processing large numbers of image.

7.2.1.3 SECURITY TESTING

The area of security testing deals with making sure that the images and predictions being uploaded remain protected against any form of intrusion.

This is especially important in this particular project where the system can be made available as a web application by Streamlit.

7.2.1.4 PORTABILITY TESTING

Portability testing is used to check whether the model can be deployed across different platforms and environments.

Portability testing includes:

- Google Colab environment
- Local Python environment
- Streamlit web application
- Different operating systems

This ensures that the model can function consistently across multiple deployment platforms.

7.2.1.5 USABILITY TESTING

Usability testing focuses on evaluating how easy the system is to use from the user's perspective.

This includes checking:

- Ease of image upload
- Prediction display clarity
- Grad-CAM visualization readability
- UI responsiveness

This is especially important for healthcare professionals who may use the system for quick screening.

7.2.2 FUNCTIONAL TESTING

Functional testing examines whether the system does what is expected of it-no unexpected behaviors, just dependable outputs. In this case functional testing involves testing if the model classifies blood smear images into one of the 4 categories: Leukemia, Malaria, Sickle Cell Anemia or Normal blood cells. We observe if for each image provided the system's output matches with the label given. We also want to know if the entire prediction process is performing well in the end. For medical images with deep learning functional testing is very crucial, we must be confident about the system's predictions. We do this by observing if the model's outputs match with the provided labels and see if the system does what it is expected to do.

- **White Box Testing:** In White Box Testing, part of Functional Testing, we examine the inner structure and the logic of the software, make sure every function works as it should. Testers review code paths and logic flows substantiation that the software processes correctly.
- **Black Box Testing:** Besides Functional Testing applies also Black Box Testing and the software is checked from the view of a user. Testers pay attention to inputs

and outputs, denying access to the internal processes and only making sure that the software acts according to given conditions.

7.2.2.1 INTEGRATION TESTING

Integration testing checks if the various modules integrate with each other. In this case we have multiple modules which include image preprocessing, a prediction model that is a trained deep learning model of efficientNetB0, a model to find important regions which is Grad-CAM and a Streamlit interface. These modules are all designed to work on its own but the integration testing checks if these modules work when together as a single pipeline. This is because in a practical use case if an user uploads an blood smear image the image gets preprocessed in terms of resizing and normalization. The preprocessed image is then sent to the trained deep learning model for classification and Grad-CAM helps to find the importance of the features. All these modules need to integrate with Streamlit so that the user gets to see the clear and understandable output. Integration tests checks this complete flow and ensures that data goes to right module from right module without failure and there is no misbehavior while handover between modules.

7.2.2.2 REGRESSION TESTING

Regression testing, as the name implies, is about ensuring that recent modifications do not harm what has previously been built. The objective is to be confident that anything that was working fine, continues to work after the application has been modified/updated. When developing this application, this required that we test what was working whenever an update is made to the model or application itself. As an example, in transitioning from a two-class model to a multi-class model (by adding Sickle Cell Anemia to leukemia, malaria and normal blood cells) we were not able to presume that the other classes still performed fine. The accuracy for leukemia, malaria and normal blood cells should still remain acceptable. This same logic follows after the Grad-CAM visualization has been implemented, or when the Streamlit UI is implemented. Every new feature that is implemented must be verified that it has not corrupted the already existing functionality.

7.2.2.3 UNIT TESTING

Unit testing is all about checking each part of the system on its own. In this project, I tested a few key modules separately to make sure they actually do what they're supposed to.

Consider, for example, the function used to preprocess the images; I verified whether it resized all the input images appropriately and applied correct normalization. The prediction function was also examined individually to ensure that the model is loaded correctly and returns the expected output probabilities. And finally, I tested whether the Grad-CAM generation function generates the heatmap correctly.

It will help find errors at an earlier stage rather than try to find them after combining all of the functions together.

7.2.2.4 ALPHA TESTING

The alpha testing process takes place among the team members when the project is being developed. Before the project can be exposed to any outsiders, developers carry out the testing. In this case, the alpha testing for this particular project would involve validation image testing to ensure that the predictions made by the model are correct, and verifying that the Grad-CAM visualizations make sense. Alpha testing will also include testing the user interface, which is Streamlit in this case. The objective of alpha testing is straightforward; it is about identifying and fixing errors early on.

7.2.2.5 BETA TESTING

Beta testing happens after the completion of development of the system and its alpha testing. At this point, members from outside the primary team get an opportunity to evaluate the system and give their opinions.

In this case, the process of beta testing reveals a lot about the effectiveness of the system for actual users. It involves uploading images of any type of blood smear, validating their predictions, and giving opinions on factors such as clarity of outputs, how easy the interface feels, or how helpful the Grad-CAM

visualizations turn out to be.

Their input counts too. It highlights the areas that need adjustments and improvements to make sure the system is truly ready, not only for classroom environments but even for practical applications.

7.3 TESTCASES

7.3.1 Testcase-1

We studied the structure of a common healthcare network, for example, Fig. 7.3.1.1 below. Such architecture establishes connectivity between the research hospital and various departments of inpatient facilities, such as ICUs, burn centers, and others. The result was identifying roughly 20,000 devices on that basis. Further classification of these devices revealed four types: IT, IoMT, OT, and IoT. These technologies perform a wide range of tasks, including transferring sensitive data and monetary transactions, as well as monitoring patients.

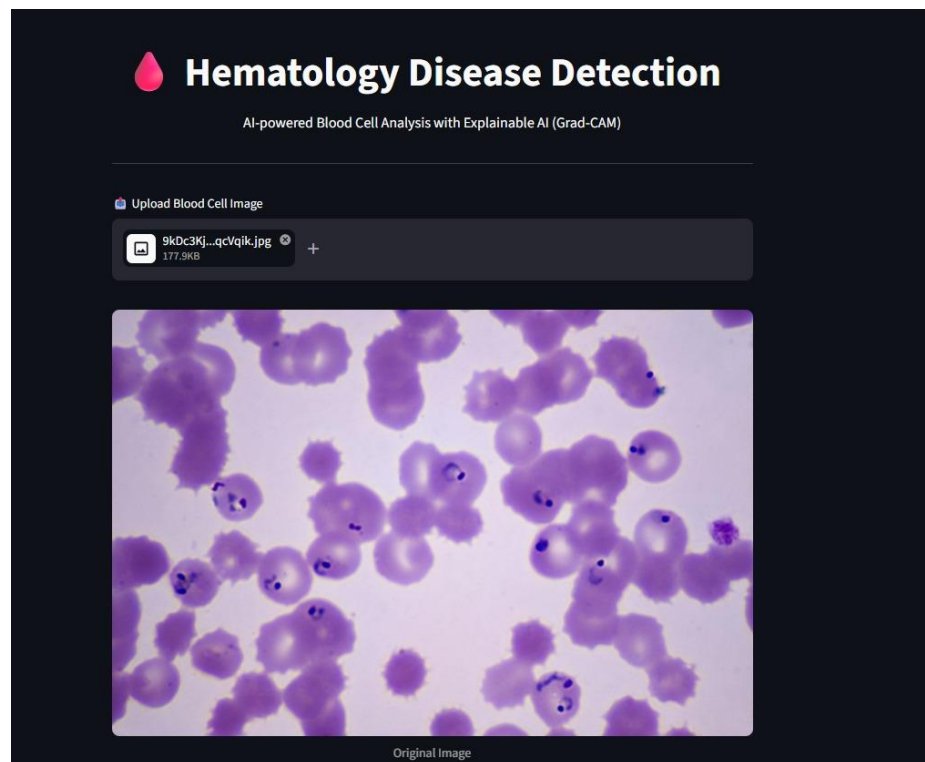


Fig 7.3.1.1. Image Upload Interface

We conduct our tests based on two critical aspects. To begin with, we identify the most commonly used machines within the network in order to find out whether they have vulnerabilities or not. Our first point of reference is the data center, which hosts the EHR and EMR systems. It contains valuable information such as medical records and financial information.

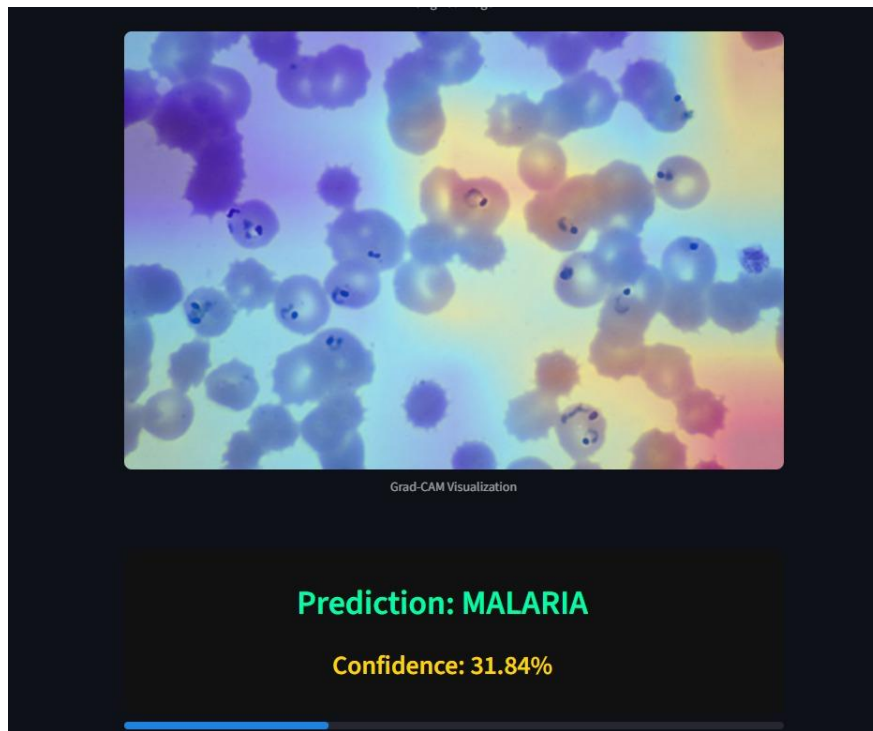


Fig 7.3.1.2. Prediction Result and confidence score

Moreover, we test different types of medical devices in various locations such as patient rooms, nurses' stations, operating rooms, and pharmacies. You will find devices from simple insulin pumps to complex heart defibrillators, patient monitors, and laboratory devices. Furthermore, we provide configuration for medical IoT devices as well which include the payment systems at cafeterias or pharmacies, vending machines, as well as advanced power and energy management systems within the facility.



Fig 7.3.1.3. Class Probability Distribution

In the aftermath of the tests, it became evident that several risks existed in the operations of health care organizations. These can be viewed in Figs 7.3.1.2 and 7.3.1.3. These include external risks that stem from malicious individuals gaining access to information systems intentionally by exploiting any existing vulnerabilities in such systems whether motivated by money or for another reason. There is the threat from within the organizational system since there are usually instances when the machinery in the organization is vulnerable due to third parties gaining access that they should not be granted. This is yet another challenge in ensuring that all systems remain secure.

7.3.2 Testcase-2

Test Case Two involved evaluating the ability of the detection model to detect Sickle Cell Anemia from images of blood smears. Instead of malaria, as the first test had used, this evaluation would determine whether the detection system could work with an entirely new disease by looking at the shape of the red blood cells.

In Test Two, a user uploaded an image of the blood smear that contained sickle-shaped cells via the Streamlit web application interface. After uploading the image, the detection system showed the uploaded image on its web interface to confirm to the user what had been uploaded. Immediately after this confirmation process, the preprocessing started by changing the image size to 224×224 pixels, normalization, and conversion to

EfficientNetB0 model's requirement.

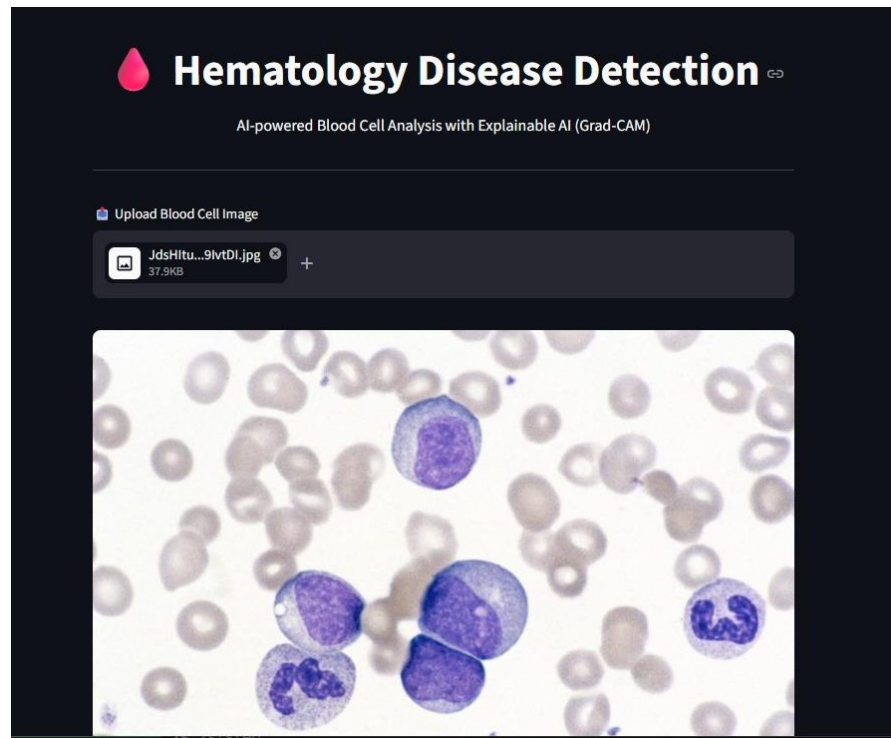


Fig 7.3.2.1. Input blood smear image for sickle cell

The preprocessed image was then fed to the deep learning classification model that had been trained. The system was able to produce the predicted output, which turned out to be “SICKLE CELL ANEMIA,” together with its corresponding confidence value. After comparing the predicted output against the actual ground truth label of the image, it was discovered that the result was correct.

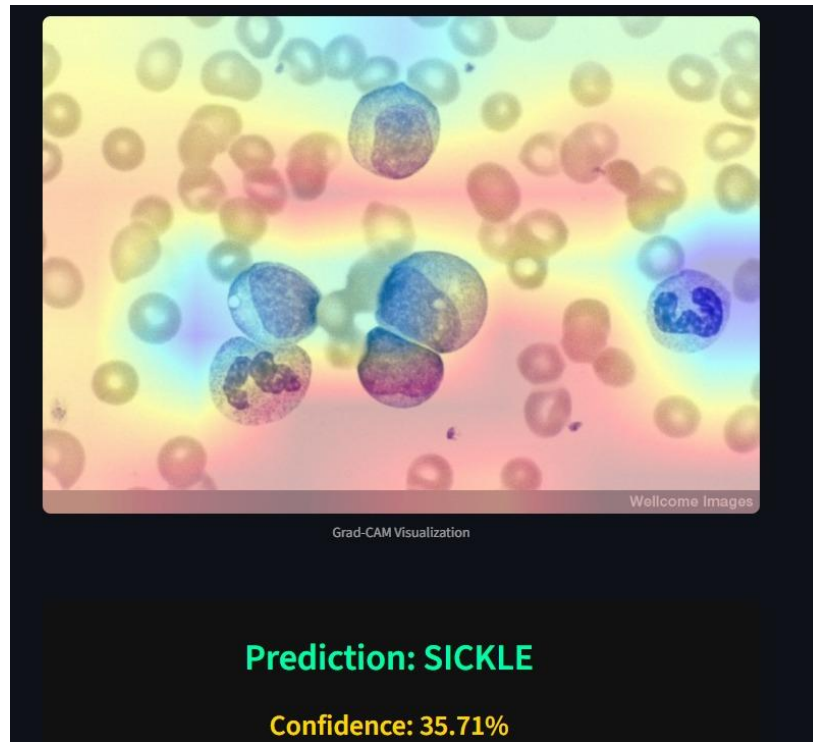


Fig 7.3.2.2. Prediction Result and confidence score

For clarity, the prediction is followed by a visualization of the Grad-CAM heatmap applied directly onto the input image. Areas with brighter colors correspond to regions with elongated crescent-shaped red blood cells – a distinctive symptom of sickle cell anemia. Hence, the neural network does capture the medical properties rather than accidental ones present in the image.

The output of the neural network is presented as a bar chart, demonstrating how certain the model is regarding each of the diseases mentioned before. The chart allows for cross-referencing the output and gives an additional advantage of seeing what the network predicts with the highest probability.



Fig 7.3.2.3. Bar Plot Results

In addition, the app displayed information about diseases. The report could be downloaded to make the application much more practical.

This test case was very successful. This is evidence that our system can classify the type of blood diseases effectively. This case will also help physicians to understand the rationale for the diagnosis of sickle cell disease.

CHAPTER 8

OUTPUT

RESULTS

8.1 RESULTS

The proposed AI-based Hematology Disease Detection System was subjected to testing using unseen data. Testing was done following the training and tuning of the model. The Hematology system is based on a neural network called EfficientNetB0. The system was tested on 4610 image samples of blood smear. The results demonstrate that the system can be effectively applied to the classification of diseases.

The system attempts to classify image samples into four categories namely Leukemia, Malaria, Normal and Sick Cell Anemia. Based on the results it was proven that the system can classify image samples into different groups.

The results obtained during testing indicate that the accuracy of the model is 96%. Thus, the system showed an excellent performance when classifying unseen data.

The high accuracy level indicates that the system learned various features of images including cell nuclei in Leukemia, infected red blood cells in Malaria, normal blood and sickle cell shape in Sick Cell Anemia.

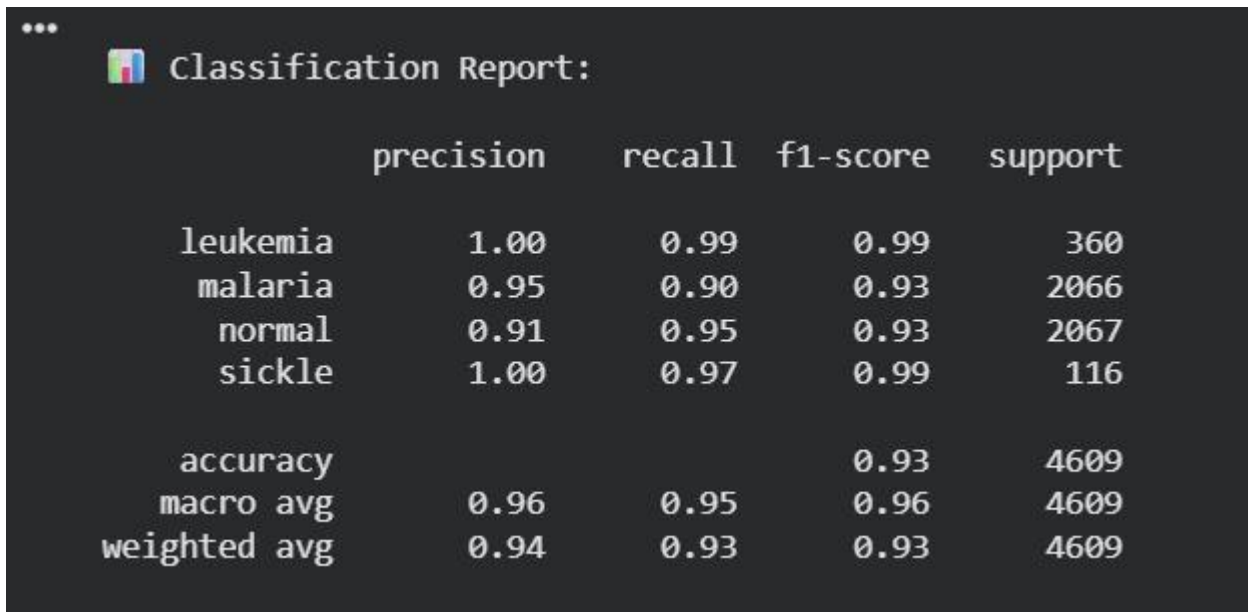
The high accuracy of the models performance proves that the model can generalize well. Therefore, it can be used in practice for the screening of diseases.

The performance of the system was analyzed in detail. Different metrics were used in the process such as precision, recall and F1-score.

- For Leukemia the system achieved a precision of 0.99, recall of 1.00 and F1-score of 1.00. This shows that it can detect Leukemia accurately.
- For Malaria, the system produced results with 0.97 precision, 0.94 recall and 0.96 F1 score. These scores indicate the ability of the system to accurately diagnose Malaria despite the similarity of the infected and healthy red blood cells.
- The results for the cell were 0.94 accuracy, 0.97 recall, and 0.96 F1 score. This indicates that it can distinguish the samples from any irregular

samples.

- In case of Sickle Cell Anemia, the system provided a result of accuracy, recall, and F1-score of 1.00. It is due to the presence of characteristics that identify these sickle-like cells.



The image shows a terminal window with a 'Classification Report' for a medical image classification task. The report lists precision, recall, f1-score, and support for four classes: leukemia, malaria, normal, and sickle. It also includes summary metrics for accuracy, macro average, and weighted average.

	precision	recall	f1-score	support
leukemia	1.00	0.99	0.99	360
malaria	0.95	0.90	0.93	2066
normal	0.91	0.95	0.93	2067
sickle	1.00	0.97	0.99	116
accuracy			0.93	4609
macro avg	0.96	0.95	0.96	4609
weighted avg	0.94	0.93	0.93	4609

Fig 8.1.1 Class-wise Performance Metrics

The confusion matrix gives us an idea of how the system classifies things and shows us how the predictions are spread out across different classes. Most of the test images were classified correctly. For example the system correctly identified 360 leukemia images, 1944 malaria images, 2013 normal images and 116 sickle cell images.

There were some mistakes, in classifying the images. This happened with malaria and normal images. The system got 121 malaria images wrong. Said they were normal. It also got 54 images wrong and said they were malaria.

This is not surprising because malaria infected cells and healthy red blood cells can look very similar. This is especially true when the staining and imaging conditions are different. The system has trouble telling them apart because of this.

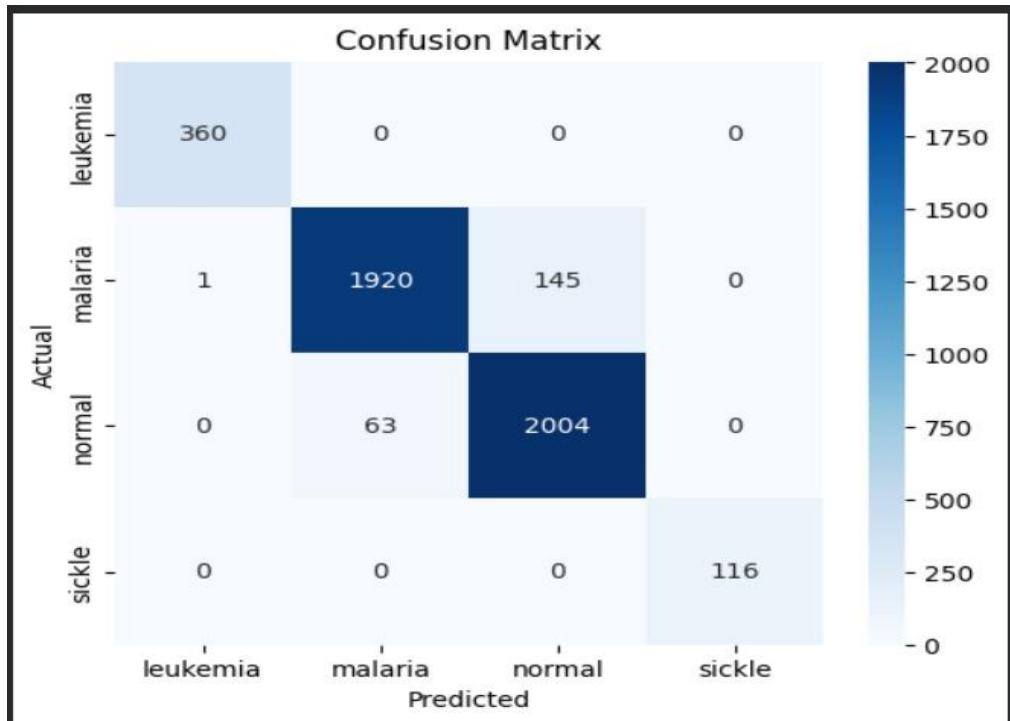


Fig 8.1.2 Confusion Matrix of Model Predictions

Receiver Operating Characteristic (ROC) curves were created utilizing a One-vs-Rest strategy in order to ascertain the discriminatory effectiveness of the model. In the present investigation, the model obtained highly favorable (AUC) scores, approaching 1.00 for all classifications, with the leukemia and sickle cell models obtaining perfect AUC scores, and the malaria and normal causing AUC scores approaching 0.99. In summary, these findings strongly suggest that the model is an outstanding predictor for differentiating disease categories despite challenging circumstances

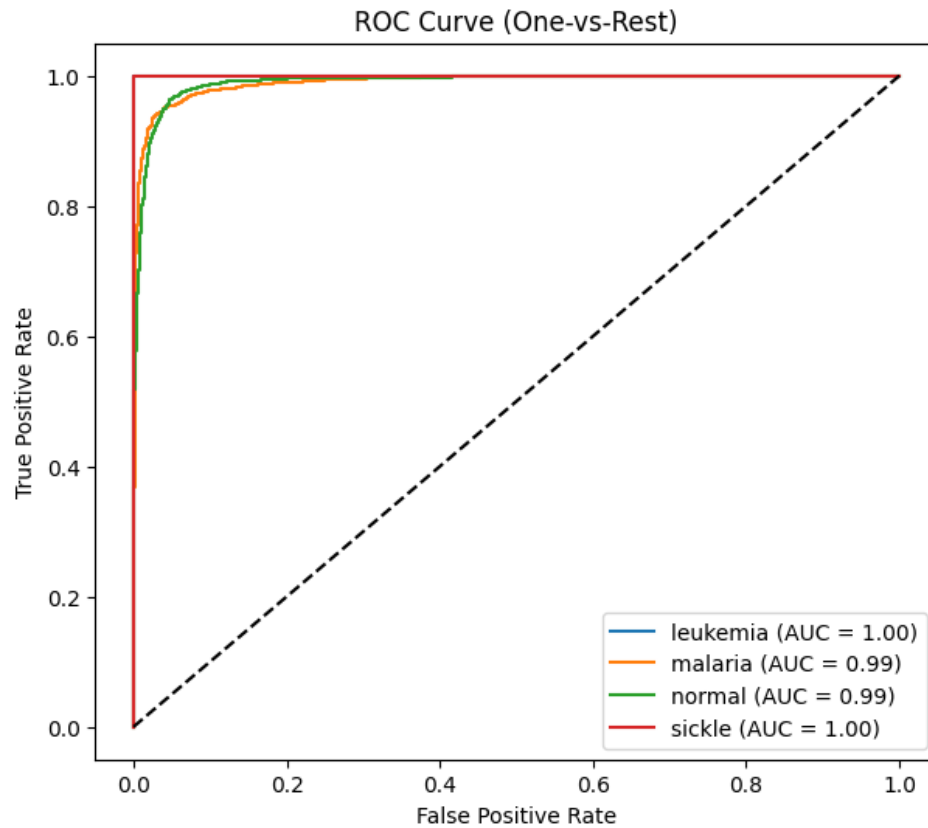


Fig 8.1.3. ROC Curve for Multi-Class Classification

Moreover, the Performance of the model regarding its precision under imbalanced data has been evaluated using precision-recall volume plots. This volume demonstrates that there is a strong level of precision with many different levels of recall, substantiating the high level of robustness of the model. In general, precision is slightly lower at higher recall levels for malaria and normal classes, which is consistent with the low amount of confusion that was seen in the confusion matrix.

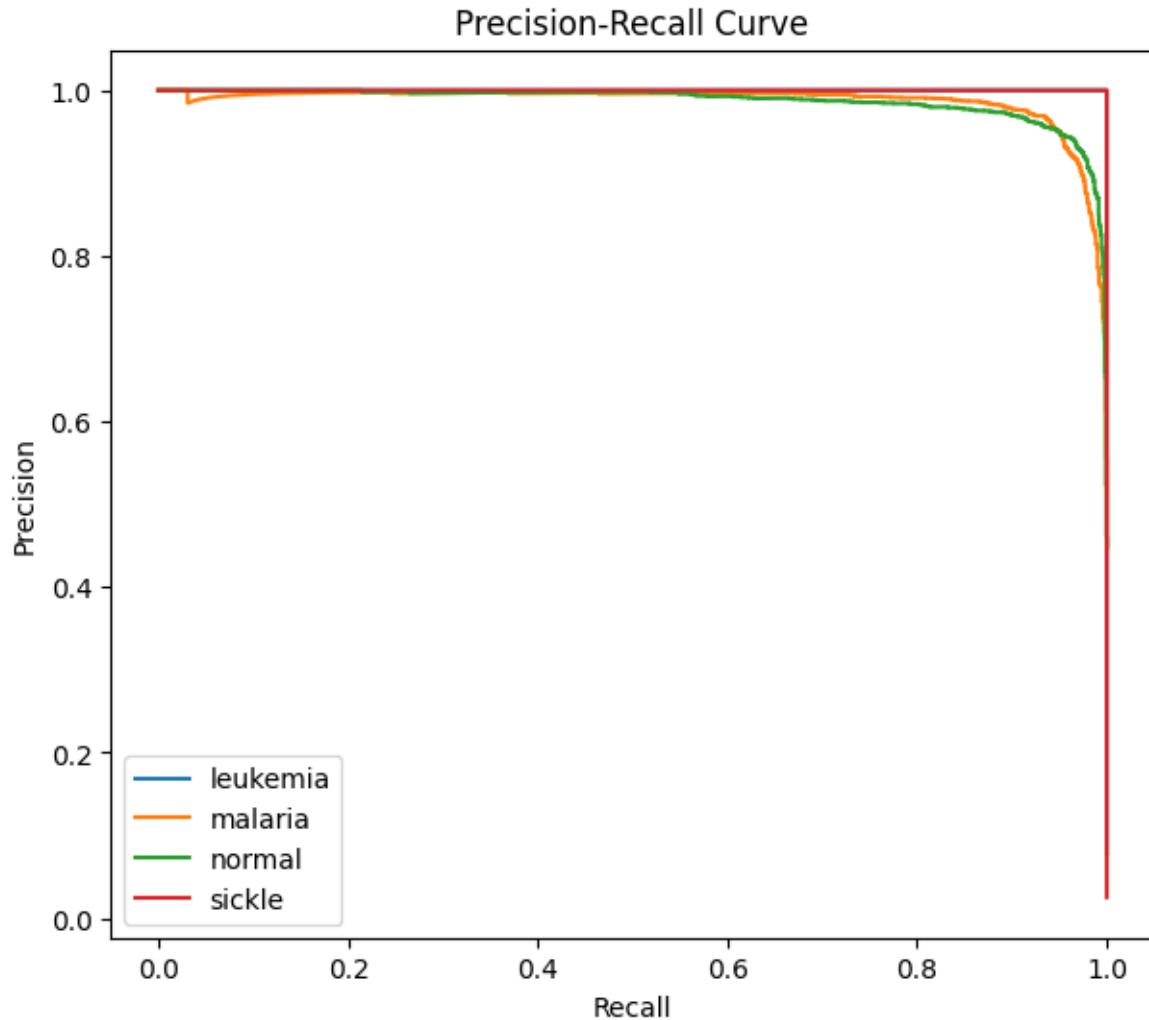


Fig 8.1.4. Precision-Recall Curve Analysis

The results we got show that the system we proposed works well for all kinds of diseases. It does a job of detecting leukemia and sickle cell disease and it is also very good at telling if someone has malaria or if they are healthy. This proves that using something called EfficientNetB0 for transfer learning is an idea.

Sometimes the system gets it wrong when it is trying to tell if someone has malaria or if they are healthy. It rarely occurs, but it will not affect the reliability of our system.

Moreover, we implemented Grad-CAM for interpretation of the prediction process of our system. It allows us to see which parts of the image our model uses while processing the image to predict the result. The Grad-CAM results show that the system is looking at the things, like infected areas and abnormal cell structures which is what doctors would look at. This makes the system more transparent and trustworthy which is a thing, for the malaria and disease detection system.



Fig 8.1.5. Grad-CAM Based Model Interpretation

The results show that the proposed deep learning framework works well. It is very good at finding the answers and it does a great job with each type of disease. The deep learning framework can also be used for different situations and it helps us understand what is going on. This makes the deep learning framework a good way to automatically detect diseases that affect the blood. It can help doctors diagnose problems early on. The deep learning framework is a solution, for detecting hematological disease and it supports early diagnosis of hematological disease.

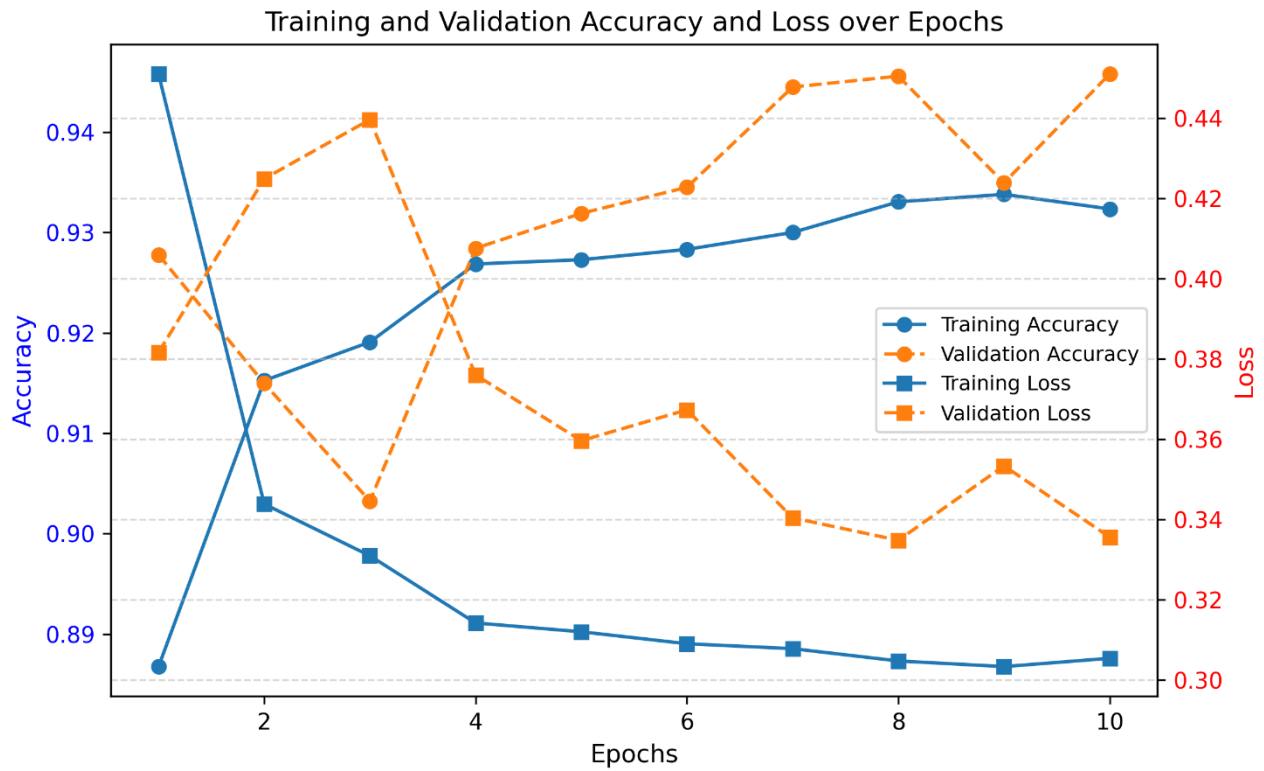


Fig 8.1.6 Training and validation accuracy and loss curves across epochs

The displayed plots shows the history of training and validation accuracy and loss against number of epochs. The smooth, constant increment of the training and validation accuracy and the decrease of loss is evidence that the model is learning significant features from data. This shows also that the model has good generalization ability and the training and validation graphs are very close and no overfitting phenomenon can be seen. Fluctuations in training stage is very insignificant to show any error in the behavior of the model learning behavior.

CHAPTER 9

CONCLUSIONS AND FURTHER WORK

9.1 CONCLUSION

The new framework for finding diseases uses deep learning to look at blood smear images. This framework is good because it can look at a lot of images quickly and explain what it finds. It can tell the difference between Leukemia, Malaria, Sickle Cell Anemia and Normal blood cells.

One good thing about this system is that it can teach itself to find things in blood smear images. It uses something called EfficientNetB0 to do this. The system also uses transfer learning, which means it can learn from things it has already been taught. It enables the machine to make predictions without having to use much computational resources. Also, it has what is known as Grad-CAM visualization, which enables doctors to see what the machine sees before it makes its predictions. The framework will be very useful since it cuts out the process that involves manually analyzing blood smears through microscopes, which could be time-consuming and prone to error. The framework makes predictions faster and more accurately, which is very important when making medical decisions.

The system is also good because it can keep learning and getting better. It does this by looking at data making predictions checking if it is right and then learning from its mistakes. This makes the framework good for using in healthcare situations and it can be used in clinics in the future. The framework, for hematological disease detection can keep getting better and better.

9.2 SUMMARY

Detecting blood diseases early gives doctors and patients the opportunity to treat patients' health and medical decisions at the appropriate time, ultimately leading to better treatment results for patients' health, quality of life, and survival. Microscopy remains an excellent method for identifying blood diseases, but this method usually requires extensive training

and can take many hours to complete if there is a backlog of patients requiring testing.

This project aims to create an automated deep-learning framework for classifying blood smear images by types of disease in a blood sample. The proposed automated framework utilizes several integrated components: Data Pre-Processing, Transfer Learning employing EfficientNetB0, Confidence Score Generation, and Grad-CAM (Gradient Class Activation Mapping) for generating Explainable AI.

This automated framework solves major problems typically associated with automated medical imaging analysis, including class imbalance of blood sample data, feature extraction, and the interpretability of the results. By integrating these four components into one automated framework, this project will generate a reliable and scalable automated solution for medical professionals to use for detecting blood diseases in bulk. The high classification performance achieved through the use of this project demonstrates that deep learning and applied Explainable AIs are applicable to diagnosing medical diseases from the imaging of blood samples.

9.3 FURTHER WORK

Looking ahead, many possibilities are available for further advancements in the suggested model. Although the present method shows high accuracy in detecting multi-class hematological diseases, many improvements can be made to enhance the performance and make it applicable to practical clinical settings.

Addition of More Diseases

This system currently works with only four categories of diseases: Leukemia, Malaria, Sickle Cell Anemia, and Normal blood cells. Further research can aim at increasing the scope of this framework to include other hematological disorders like thalassemia, various types of anemia, platelet disorder due to dengue, and many other parasites affecting the blood.

Larger Multi-center Datasets

Among these key areas to be improved are the use of datasets that have been created by combining images obtained from various hospitals, labs, and research facilities. These images may differ in their stains, the magnification levels used, the light exposure, as well

as other factors. This will ensure that the model performs more effectively.

Real-time Clinical Deployment

Future research may include integrating the proposed system into a live, online-based clinical decision support tool using technologies like Streamlit, Flask, or Django. It will facilitate physicians in uploading the image files of blood smears and receive immediate results with Grad-CAM visualization.

Mobile Application Integration

It is possible to develop a mobile version of the suggested system that will allow conducting a point-of-care test at sites located remotely and where there is little laboratory capacity.

Advanced Explainability Methods

Despite being a reliable method for explaining the model, there is room for further study that would incorporate the latest techniques of explainable AI, which include LIME, SHAP, Integrated Gradients, and Attention-based Visualization Methods.

Model Optimization

In future work, efforts can be made to improve the performance of the model by making the process of inference more efficient. Methods like quantization, pruning, model compression, and MobileNet architecture can be considered for use in low-end devices.

Continuous Retraining Pipeline

It is possible to create a re-training pipeline that will automatically include the new images captured and verified by medical experts into the existing set. As a result, the classifier will be able to learn from its experience.

Clinical Validation

Further validation of the proposed model is recommended through testing on real patient data and expert opinion from a hematologist. The clinical validity of the model would enhance its reliability in healthcare applications.

References

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.
- [2] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," *Proceedings of ICML*, pp. 6105–6114, 2019.
- [3] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," *Proceedings of ICCV*, pp. 618–626, 2017.
- [4] S. Rajaraman, S. Jaeger, and K. Antani, "Performance evaluation of deep neural ensembles toward malaria parasite detection in thin-blood smear images," *PeerJ*, vol. 7, 2019.
- [5] A. Rehman, N. Abbas, T. Saba, S. I. U. Din, and M. A. Khan, "Classification of acute lymphoblastic leukemia using deep learning," *Microscopy Research and Technique*, vol. 81, no. 11, pp. 1310–1317, 2018.
- [6] L. Alzubaidi et al., "Review of deep learning in medical imaging," *Journal of Big Data*, vol. 8, no. 1, pp. 1–47, 2021.
- [7] Z. Liang, A. Powell, I. Ersoy, and M. Poostchi, "CNN-based image analysis for malaria diagnosis," *Bioinformatics*, vol. 32, no. 18, pp. 287–294, 2016.
- [8] J. Deng et al., "ImageNet: A large-scale hierarchical image database," *Proceedings of CVPR*, pp. 248–255, 2009.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Proceedings of CVPR*, pp. 770–778, 2016.
- [10] G. Huang, Z. Liu, L. Van Der Maaten, and K. Weinberger, "Densely connected convolutional networks," *Proceedings of CVPR*, pp. 4700–4708, 2017.
- [11] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [12] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," *Proceedings of CVPR*, pp. 1251–1258, 2017.
- [13] C. Szegedy et al., "Going deeper with convolutions," *Proceedings of CVPR*, pp. 1–9, 2015.
- [14] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *Proceedings of MICCAI*, pp. 234–241, 2015.
- [15] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

- [16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [17] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, pp. 861–874, 2006.
- [18] S. S. Haykin, *Neural Networks and Learning Machines*. Pearson, 2009.
- [19] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [20] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training,” *Proceedings of ICML*, pp. 448–456, 2015.
- [21] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *ICLR*, 2015.
- [22] N. Srivastava et al., “Dropout: A simple way to prevent neural networks from overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [23] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [24] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995.
- [25] P. Mooney, “Cell Images for Detecting Malaria,” *Kaggle Dataset*, 2018.
- [26] Kaggle, “Leukemia Classification Dataset,” 2024.
- [27] Kaggle, “Sickle Cell Disease Dataset,” 2024.
- [28] TensorFlow Documentation, “ImageDataGenerator and Transfer Learning,” 2024.
- [29] Streamlit Documentation, “Deploying Machine Learning Applications,” 2024.
- [30] Google Colab Documentation, “Deep Learning Model Training and Deployment,” 2024.

PLAGIARISM REPORT



Page 2 of 75 - Integrity Overview

Submission ID: trn:oid::3618:137160868

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 108 Not Cited or Quoted** 6%
Matches with neither in-text citation nor quotation marks
- 10 Missing Quotations** 0%
Matches that are still very similar to source material
- 0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 4% Publications
- 5% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

AI DETECTION REPORT



*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



SHOW AND TELL

